G / C
GARAGE COFFEE

Manual técnico completo

Java Swing · POO · JDBC/MySQL · Spring Boot · Android/Kotlin · Jetpack Compose

Explicado paso a paso, con palabras sencillas y capturas del código real.

Versión revisada: 20 de octubre de 2026

Cómo usar este manual

Imagina que el programa es un restaurante de verdad. Las pantallas son quienes atienden; los servicios son los encargados que conocen las reglas; los DAO son archivistas que saben hablar SQL; MySQL es el archivo; Spring es la ventanilla para Android; y la app móvil es la libreta del mesero.

No necesitas memorizar todo. Primero lee el mapa y los flujos. Después busca la clase que estés modificando.

Contenido

Cómo usar este manual 2

Contenido 3

1. El proyecto en una sola mirada 6

Flujo de una acción 6

2. POO explicada como piezas de juguete 7

Dónde aparece POO 7

2.1 Mapa completo de herencia 8

2.2 Pedido y PedidoLocal 9

2.3 DatosRestaurante y DatosMySQL 10

2.4 Herencia en Swing 11

2.5 Herencia de frameworks e interfaces 12

2.6 Encapsulamiento 13

2.7 Composición e inyección 14

2.8 Polimorfismo, genéricos y callbacks 15

3. Modelos, servicios y DAO de Java 16

Modelos, servicios, datos y DAO 16

4. La base de datos y JDBC 18

Transacciones, DAO y SQL preparado 19

5. Swing: la aplicación de escritorio 20

6. Spring: el puente para Android 22

Rutas, objetos y seguridad 22

7. Android, Kotlin y Jetpack Compose 24

Estado, cliente HTTP y pantallas 25

8. Flujos completos del restaurante 27

9. Configuración, compilación y ejecución 27

10. Errores comunes explicados 28

Apéndice A · Funciones declaradas en el código 29

Apéndice B · Glosario sencillo 50

Índice de clases y archivos

Las páginas del apéndice explican las funciones declaradas dentro de cada archivo.

AccesoActivity 29
ApiCliente 29
InicioActivity 29
SesionApp 31
PantallaAcceso 31
PantallaInicio 31
TemaGarageCoffee 31
BackendApplication 31
ConfiguracionDatos 31
ConsultaRestauranteService 32
ErroresApi 32
OperacionRestauranteService 32
RestauranteApi 33
SeguridadApi 33
Proyecto2Progra2 33
RestauranteController 33
CajaDAO 34
CategoriaDAO 34
ControlCambiosDAO 34
DetallePedidoDAO 34
EmpleadoDAO 34
MesaDAO 35
PagoDAO 35
PedidoDAO 35
ProductoDAO 36
ConexionDB 36
Caja 36
Categoria 37
DetallePedido 37
Empleado 38
Mesa 38
Pago 39
Pedido 39
PedidoLocal 40

Índice de clases y archivos

Producto 40
AtencionService 41
CatalogoService 41
Claves 41
DatosMySQL 41
FinanzasService 42
PedidoService 42
SesionService 43
Validacion 43
AperturaCajaForm 43
AtencionPanel 44
CajaPanel 44
CantidadForm 44
CatalogoPanel 44
CategoriaForm 44
CierreCajaForm 45
CobroForm 45
CocinaPanel 45
DashboardFrame 45
EmpleadoForm 45
EmpleadosPanel 46
Formulario 46
InicioPanel 46
LineaPedidoForm 47
LoginFrame 47
MesaForm 47
PanelListado 47
PanelModulo 48
PedidoDatosForm 48
PedidoDialog 48
PedidosPanel 48
ProductoForm 49
TextoDialog 49
Ui 49

1. El proyecto en una sola mirada

Garage Coffee tiene tres programas que comparten una misma idea de restaurante. Swing es el escritorio principal; Spring es un servidor; Android es el cliente móvil. Swing y Spring usan la misma base MySQL. Android nunca se conecta directamente a MySQL.

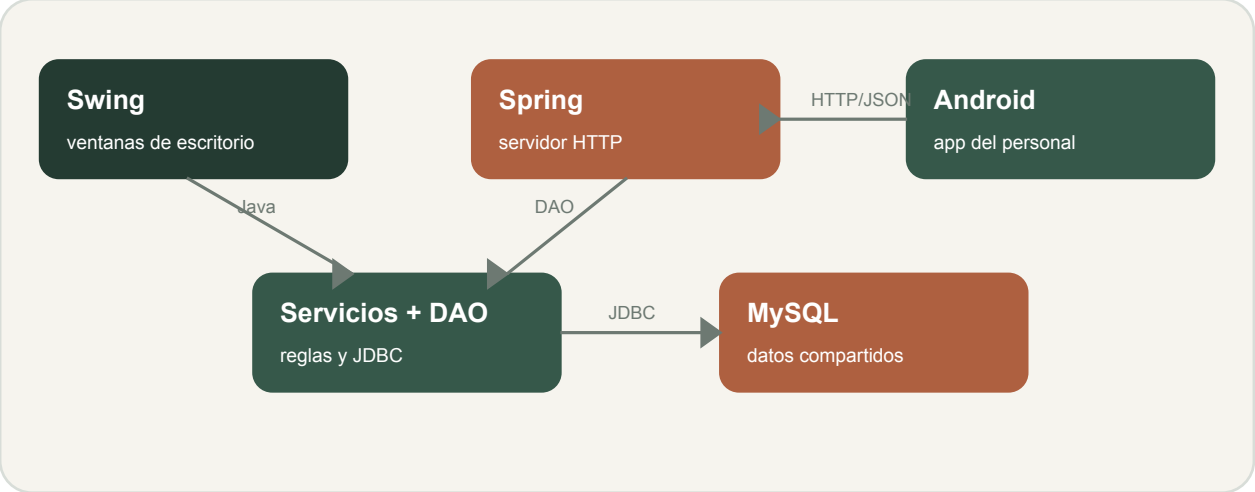


Figura 1. Arquitectura real del proyecto.

- **Swing:** inicia sesión, administra productos/mesas/empleados, toma pedidos, controla cocina y cobra.
- **Spring:** recibe HTTP, autentica por rol y ejecuta consultas u operaciones usando DAO.
- **Android:** muestra mesas y menú, crea o amplía pedidos, entrega y cobra mediante Spring.
- **MySQL:** conserva datos aunque las aplicaciones se cierren.

Flujo de una acción

- 1. La persona pulsa un botón o escribe datos.
- 2. La pantalla convierte esa acción en una llamada clara.
- 3. El servicio revisa permisos, estados y datos.
- 4. DatosMySQL y la DAO ejecutan SQL dentro de una transacción.
- 5. MySQL guarda o devuelve filas.
- 6. El resultado vuelve a la pantalla y se muestra.

2. POO explicada como piezas de juguete

POO significa Programación Orientada a Objetos. Una clase es un molde; un objeto es una pieza creada con ese molde. `Mesa` es el molde y la mesa número 4 es un objeto.

Idea	Explicación para empezar	Ejemplo real
Clase/objeto	Molde y cosa creada con él	Mesa → mesa 4
Encapsulación	El objeto cuida sus datos	Mesa.ocupar() impide ocupar una mesa fuera de servicio
Abstracción	Mostrar solo lo necesario	PedidoService dice guardar pedido sin enseñar SQL
Herencia	Un molde reutiliza otro	PedidoLocal extends Pedido
Polimorfismo	Misma idea, distintas formas	DatosRestaurante permite trabajar mediante un contrato general
Composición	Un objeto contiene otros	Pedido contiene DetallePedido
Enum	Lista cerrada de opciones	EstadoPedido evita estados inventados
Interfaz/callback	Promesa de funciones	Respuesta<T> exige correcto o error

01_herencia · src/main/java/com/garagecoffee/model/PedidoLocal.java

```java
package com.garagecoffee.model;

// representa un pedido de mesa.
public class PedidoLocal extends Pedido {
    private Mesa mesa;

    public PedidoLocal(int id, String observaciones, Mesa mesa) {
        super(id, observaciones);
        if (mesa == null) throw new IllegalArgumentException("La mesa es obligatoria.");
        this.mesa = mesa;
    }

    public Mesa getMesa() { return mesa; }
    public void setMesa(Mesa mesa) {
        if (mesa == null) throw new IllegalArgumentException("La mesa es obligatoria.");
        this.mesa = mesa;
    }
```

Figura 2. Herencia: PedidoLocal reutiliza Pedido y agrega la mesa.

Dónde aparece POO

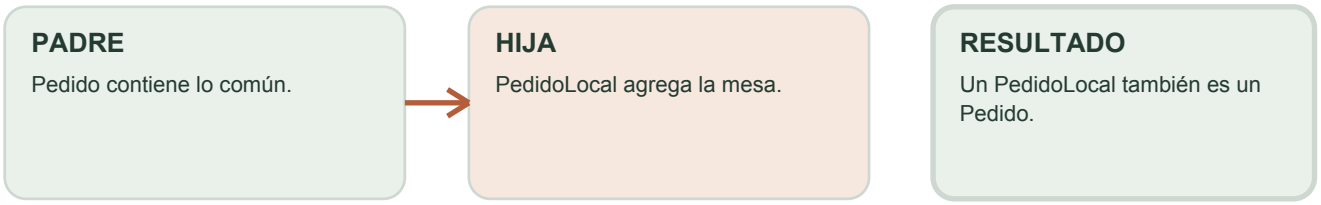
- Los modelos encapsulan datos con campos privados y getters/setters.
- Pedido es abstracta: no se crea un pedido sin saber qué tipo es.
- PedidoLocal hereda fecha, estado, total y detalles.
- RestauranteController compone varios servicios que colaboran.
- Los servicios reciben objetos y les piden comportamiento: ocupar, liberar, cancelar o calcular.
- PanelListado<T> y Respuesta<T> usan genéricos para trabajar con diferentes tipos sin duplicar clases.

2.1 Mapa completo de herencia

Una clase padre guarda lo que varias clases tienen en común. Una clase hija recibe ese trabajo y agrega lo que la hace especial. La flecha se lee: la hija hereda de la clase padre.

Clase padre	Clases hijas directas	Qué reciben
Pedido	PedidoLocal	id, fecha, estado, total, observaciones y detalles
DatosRestaurante	DatosMySQL	listas compartidas y operaciones abstractas
PanelModulo	InicioPanel, EmpleadosPanel, CatalogoPanel, AtencionPanel, PedidosPanel, CocinaPanel y CajaPanel	ser un JPanel y poder recargar listados
Formulario	AperturaCajaForm, CantidadForm, CategoriaForm, CierreCajaForm, CobroForm, EmpleadoForm, LineaPedidoForm, MesaForm, PedidoDatosForm y ProductoForm	campos, lectura, conversión, validación y guardado modal
JFrame	LoginFrame y DashboardFrame	comportamiento de una ventana principal
JDialog	PedidoDialog y TextoDialog	comportamiento de una ventana modal
ComponentActivity	AccesoActivity e InicioActivity	ciclo de vida y pantalla Android
Application	SesionApp	objeto global del proceso Android

Cómo leer una relación



2.2 Pedido es el padre; PedidoLocal es la hija

Pedido es abstracta: sirve como plantilla y no se crea directamente. PedidoLocal usa `extends Pedido`, llama a `super(...)` y añade una Mesa. Esta es la herencia propia más clara del modelo.

src/main/java/com/garagecoffee/model/Pedido.java

```java
// contiene los datos de un pedido.
public abstract class Pedido {
    private int id;
    private LocalDateTime fechaCreacion;
    private EstadoPedido estado;
    private BigDecimal total;
    private String observaciones;
    private final List<DetallePedido> detalles = new ArrayList<>();

    protected Pedido(int id, String observaciones) {
        this.id = id;
        this.fechaCreacion = LocalDateTime.now();
        this.estado = EstadoPedido.PENDIENTE;
        this.total = BigDecimal.ZERO;
        this.observaciones = observaciones;
    }

    public BigDecimal calcularTotal() {
        total = detalles.stream().map(DetallePedido::calcularSubtotal)
                .reduce(BigDecimal.ZERO, BigDecimal::add);
        return total;
    }

    public void agregarDetalle(DetallePedido detalle) {
        if (detalle == null) throw new IllegalArgumentException("El detalle del pedido es obligatorio.");
        detalles.add(detalle);
        calcularTotal();
    }

    public void cancelar() { estado = EstadoPedido.CANCELADO; }
```

src/main/java/com/garagecoffee/model/PedidoLocal.java

```java
public class PedidoLocal extends Pedido {
    private Mesa mesa;

    public PedidoLocal(int id, String observaciones, Mesa mesa) {
        super(id, observaciones);
        if (mesa == null) throw new IllegalArgumentException("La mesa es obligatoria.");
        this.mesa = mesa;
    }

    public Mesa getMesa() { return mesa; }
    public void setMesa(Mesa mesa) {
        if (mesa == null) throw new IllegalArgumentException("La mesa es obligatoria.");
        this.mesa = mesa;
    }
```

Parte	Qué ocurre
abstract class Pedido	Impide crear un pedido incompleto; define la base común.
extends Pedido	Declara que PedidoLocal es una versión concreta de Pedido.
super(id, observaciones)	Ejecuta el constructor del padre antes de guardar la mesa.
List<DetallePedido>	Es composición: el pedido contiene sus líneas.
Mesa mesa	Es asociación: el pedido local conoce la mesa atendida.
getDetalles() inmodificable	Protege la lista para que nadie la cambie sin usar agregarDetalle.

2.3 DatosRestaurante es el contrato de datos

DatosRestaurante es la clase padre abstracta. Declara qué necesita el negocio, pero no decide cómo se guarda. DatosMySQL es la hija concreta y cumple esas operaciones usando DAO y JDBC.

src/main/java/com/garagecoffee/service/DatosRestaurante.java

```
9  public abstract class DatosRestaurante {
10     final List<Empleado> empleados = new ArrayList<>();
11     final List<Categoria> categorias = new ArrayList<>();
12     final List<Producto> productos = new ArrayList<>();
13     final List<Mesa> mesas = new ArrayList<>();
14     final List<Pedido> pedidos = new ArrayList<>();
15     final List<Pago> pagos = new ArrayList<>();
16     final List<Caja> cajas = new ArrayList<>();
17     abstract int siguienteId(Class<?> tipo);
18     public abstract void recargar();
19     abstract void guardar(Object registro);
```

src/main/java/com/garagecoffee/service/DatosMySQL.java

```
11 public class DatosMySQL extends DatosRestaurante {
12     private final ControlCambiosDAO cambios = new ControlCambiosDAO();
13     private long revision;
14     private boolean escribiendo;
15     private boolean disponible;
16     public DatosMySQL() { recargar(); }
17     // mysql asigna los identificadores.
18     @Override int siguienteId(Class<?> tipo) { return 0; }
19     @Override public void recargar() {
20         disponible = false;
21         try {
22             ConexionDB.transaccion(() -> { cargar(); return null; });
23             disponible = true;
24         } catch (SQLException e) {
25             throw new IllegalStateException("No se pudo cargar MySQL. Revisa la conexión y ejecuta garage_coffee.sql si falta e…
26         }
27     }
28     @Override <T> T transaccion(Supplier<T> accion) {
29         if (ConexionDB.enTransaccion()) return accion.get();
30         if (!disponible) throw new IllegalStateException("Actualiza los datos antes de continuar.");
31         escribiendo = false;
32         try {
33             T resultado = ConexionDB.transaccion(() -> {
34                 long actual = cambios.revision(true);
35                 if (actual != revision) throw new SQLException("Otra sesión modificó los datos. Pulsa Actualizar y vuelve a abr…
36                 T valor = accion.get();
```

Elemento	Explicación
DatosRestaurante datos	El controlador depende del padre, no de una implementación rígida.
new DatosMySQL()	La aplicación normal entrega al controlador la hija que usa MySQL.
@Override recargar	DatosMySQL reemplaza la operación abstracta con una versión real.
guardar(Object)	El patrón switch elige el DAO según la clase concreta del objeto.
Beneficio	Los servicios usan el mismo contrato y no necesitan conocer JDBC.

2.4 Herencia en Swing

Swing ya trae clases padre. El proyecto añade padres propios para evitar repetir la misma tabla, formulario o recarga en cada pantalla.

Cadena de herencia	Hijas concretas	Trabajo reutilizado
JPanel → PanelModulo	InicioPanel, EmpleadosPanel, CatalogoPanel, AtencionPanel, PedidosPanel, CocinaPanel y CajaPanel	registrar listados y recargarlos juntos
JPanel → Formulario	10 formularios de caja, catálogo, empleado, mesa y pedido	registrar campos, leer texto/números/fechas y mostrar validaciones
JPanel → PanelListado<T>	No tiene hijas: se reutiliza por composición	tabla genérica, búsqueda, botones y selección
JFrame	LoginFrame y DashboardFrame	ventana, título, cierre y contenido
JDialog	PedidoDialog y TextoDialog	diálogo modal encima de otra ventana

Ejemplo: PanelModulo

src/main/java/com/garagecoffee/view/PanelModulo.java

```
8  public abstract class PanelModulo extends JPanel {
9      private final List<PanelListado<?>> listados = new ArrayList<>();
10     protected void registrar(PanelListado<?> listado) { listados.add(listado); }
11     public void recargar() { listados.forEach(PanelListado::recargar); }
12
13 }
```

DashboardFrame guarda todos los módulos en un Map<String, PanelModulo>. Gracias al padre, puede llamar `recargar()` sin preguntar si el objeto real es CatalogoPanel, CajaPanel o CocinaPanel. Eso es polimorfismo.

2.5 Padres de frameworks e interfaces

No toda herencia nace dentro del proyecto. Swing y Android entregan clases padre; Java también permite cumplir interfaces, que son promesas de métodos.

Clase del proyecto	Padre o interfaz	Métodos que adapta
Formulario	JPanel + Scrollable	tamaño del área, desplazamiento y seguimiento del ancho
Recurso	AutoCloseable	close(), llamado automáticamente por try-with-resources
ConexionDB.Operacion<T>	@FunctionalInterface	ejecutar(), usado por una lambda dentro de una transacción
ApiCliente.Respuesta<T>	interface genérica	correcto(T) y error(String)
ApiCliente.Operacion<T>	interface privada	ejecutar() para una tarea de red
AccesoActivity	ComponentActivity	onCreate()
InicioActivity	ComponentActivity	onCreate() y onDestroy()
SesionApp	Application	onCreate() y una sesión compartida

src/main/java/com/garagecoffee/database/ConexionDB.java

```
34      public static final class Recurso implements AutoCloseable {
35          public final Connection conexion;
36          private final boolean propia;
37          private Recurso(Connection conexion, boolean propia) { this.conexion = conexion; this.propia = propia; }
38          @Override public void close() throws SQLException { if (propia) conexion.close(); }
39      }
40      public static void confirmar(Connection conexion) throws SQLException { if (actual.get() != conexion) conexion.commit()…
41      public static void revertir(Connection conexion) throws SQLException { if (actual.get() != conexion) conexion.rollback(…
42      public static boolean enTransaccion() { return actual.get() != null; }
43      @FunctionalInterface public interface Operacion<T> { T ejecutar() throws SQLException; }
44      public static <T> T transaccion(Operacion<T> operacion) throws SQLException {
45          if (enTransaccion()) return operacion.ejecutar();
46          try (Connection conexion = getConnection()) {
47              conexion.setAutoCommit(false);
48              actual.set(conexion);
49              try {
50                  T resultado = operacion.ejecutar();
51                  conexion.commit();
52                  return resultado;
```

- La anotación @Override avisa que un método reemplaza o cumple uno heredado.

2.6 Encapsulamiento: cada objeto cuida sus datos

Encapsular significa guardar los atributos como `private` y permitir cambios mediante métodos controlados. Así el objeto puede impedir estados imposibles.

src/main/java/com/garagecoffee/model/DetallePedido.java

```java
    private int id;
    private int pedidoId;
    private int cantidad;
    private BigDecimal precioUnitario;
    private BigDecimal subtotal;
    private Producto producto;

    public DetallePedido() {
        this.precioUnitario = BigDecimal.ZERO;
        this.subtotal = BigDecimal.ZERO;
    }

    public DetallePedido(
            int id,
            int cantidad,
            BigDecimal precioUnitario,
            Producto producto) {
        this();
        this.id = id;
        validarCantidad(cantidad);
        validarPrecioUnitario(precioUnitario);
        this.cantidad = cantidad;
        this.precioUnitario = precioUnitario;
        this.producto = producto;
        this.subtotal = precioUnitario.multiply(BigDecimal.valueOf(cantidad));
    }

    public BigDecimal calcularSubtotal() {
        if (cantidad <= 0 || precioUnitario == null) {
            subtotal = BigDecimal.ZERO;
        } else {
            subtotal = precioUnitario.multiply(BigDecimal.valueOf(cantidad));
        }
        return subtotal;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

```

Clase	Qué protege	Método que controla la regla
DetallePedido	cantidad, precio y subtotal	setCantidad, setPrecioUnitario y calcularSubtotal
Pedido	detalles y total	agregarDetalle y calcularTotal
Mesa	capacidad y estado	ocupar, liberar, cambiarEstado y validarCapacidad
Producto	precio, disponibilidad y categoría	setPrecio, cambiarDisponibilidad y setCategoria
Empleado	cuenta, rol, clave y activo	setters y estaActivo
ApiCliente	autorización y sesión móvil	ingresar y salir

Ejemplo infantil: la caja de galletas está cerrada. No metes la mano directamente; pides una galleta por el método permitido. Los atributos `private` son la caja cerrada.

2.7 Composición, asociación e inyección

Los objetos también colaboran porque contienen o conocen otros objetos. Esto no significa herencia: un Pago no es una Caja; solamente tiene una Caja relacionada.

Objeto	Contiene o conoce	Tipo de relación y uso
Pedido	List<DetallePedido>	composición: las líneas forman el pedido
DetallePedido	Producto	asociación: identifica qué se vendió
PedidoLocal	Mesa	asociación: indica dónde se atiende
Producto	Categoria	asociación: clasifica el producto
Pago	Pedido y Caja	asociación: enlaza la cuenta cobrada y la jornada
RestauranteController	cinco servicios	composición: reúne el trabajo de la aplicación Swing
Servicios	DatosRestaurante y SesionService	inyección por constructor: reciben colaboradores ya creados
DashboardFrame	Map<String, PanelModulo>	composición: conserva las pantallas abiertas
Spring services	DAO	inyección de dependencias mediante el constructor
SesionApp	ApiCliente	composición: conserva un cliente HTTP compartido

src/main/java/com/garagecoffee/controller/RestauranteController.java

```
6  public class RestauranteController {
7      private final DatosRestaurante datos;
8      private final SesionService sesion;
9      private final CatalogoService catalogo;
10     private final AtencionService atencion;
11     private final PedidoService pedidos;
12     private final FinanzasService finanzas;
13     public RestauranteController() { this(new DatosMySQL()); }
14     public RestauranteController(DatosRestaurante datos) {
15         this.datos = datos;
16         sesion = new SesionService(datos);
17         catalogo = new CatalogoService(datos, sesion);
18         atencion = new AtencionService(datos, sesion);
19         pedidos = new PedidoService(datos, sesion, catalogo);
20         finanzas = new FinanzasService(datos, sesion, pedidos);
21     }
22     public void recargar() { datos.recargar(); }
23     public SesionService sesion() { return sesion; }
24     public CatalogoService catalogo() { return catalogo; }
25     public AtencionService atencion() { return atencion; }
```

La inyección hace visible la dependencia: el constructor dice qué necesita la clase. Esto permite reutilizar el mismo DatosRestaurante en todos los servicios y evita que cada pantalla abra su propia conexión.

2.8 Polimorfismo, genéricos y callbacks

Polimorfismo significa tratar objetos diferentes mediante un mismo padre o interfaz. Genéricos significa reutilizar una clase sin perder el tipo. Un callback es un objeto que responde cuando termina otra tarea.

Lugar exacto	Qué concepto usa	Qué permite
RestauranteController(DatosRestaurante)	polimorfismo	recibir DatosMySQL mediante el tipo padre
List<Pedido>	polimorfismo	guardar PedidoLocal como Pedido
Map<String, PanelModulo>	polimorfismo	manejar siete paneles con recargar()
switch (registro) en DatosMySQL	polimorfismo por tipo	elegir DAO según Empleado, Producto, Mesa, Pedido, Pago o Caja
PanelListado<T>	genéricos	usar la misma tabla con Mesa, Producto, Pedido o Empleado
ApiCliente.Respuesta<T>	interfaz + genéricos	devolver String, JSONArray o JSONObject con la misma forma
Supplier, Function, Consumer y Runnable	interfaces funcionales	pasar comportamiento como dato mediante lambdas
@Composable	composición declarativa	construir una pantalla uniendo funciones pequeñas
records y data classes	objetos de datos	transportar información sin repetir getters y constructores

Recorrido real de una acción

1. CatalogoPanel usa un PanelListado<Producto>.
2. El botón entrega un Producto a CatalogoService.
3. CatalogoService trabaja con el padre DatosRestaurante.
4. El objeto real es DatosMySQL, por eso se ejecuta su guardar().
5. DatosMySQL reconoce Producto y delega a ProductoDAO.
6. ProductoDAO prepara SQL y MySQL conserva el cambio.

Idea final

Herencia comparte una identidad; composición conecta objetos; encapsulamiento protege datos; abstracción separa el qué del cómo; polimorfismo permite usar distintas clases bajo el mismo contrato.

02_controlador · src/main/java/com/garagecoffee/controller/RestauranteController.java

```java
    private final AtencionService atencion;
    private final PedidoService pedidos;
    private final FinanzasService finanzas;
    public RestauranteController() { this(new DatosMySQL()); }
    public RestauranteController(DatosRestaurante datos) {
        this.datos = datos;
        sesion = new SesionService(datos);
        catalogo = new CatalogoService(datos, sesion);
        atencion = new AtencionService(datos, sesion);
        pedidos = new PedidoService(datos, sesion, catalogo);
        finanzas = new FinanzasService(datos, sesion, pedidos);
    }
    public void recargar() { datos.recargar(); }
    public SesionService sesion() { return sesion; }
    public CatalogoService catalogo() { return catalogo; }
    public AtencionService atencion() { return atencion; }
    public PedidoService pedidos() { return pedidos; }
    public FinanzasService finanzas() { return finanzas; }
}
```

Figura 3. Composición: un controlador reúne los servicios que comparten datos y sesión.

3. Modelos, servicios y DAO de Java

Inicio y coordinación

Proyecto2Progra2

Punto de entrada de Swing. `main` prepara la apariencia, crea el controlador con MySQL y abre el acceso.

RestauranteController

Construye y comparte sesión, catálogo, atención, pedidos y finanzas. `recargar` vuelve a leer MySQL; sus demás métodos entregan cada servicio.

Modelos

Empleado

Ficha de una cuenta. Sus constructores crean la ficha; los getters leen datos y los setters los cambian.

RolEmpleado

Lista cerrada de roles: administrador, mesero, cocina y cajero.

Categoria / TipoCategoria

Agrupa productos y limita el tipo a comida, bebida o extra.

Producto

Representa algo vendible. Además de getters/setters, `ActualizarPrecio` cambia el precio y `cambiarDisponibilidad` invierte su estado.

Mesa / EstadoMesa

Guarda número, capacidad y estado. `ocupar`, `liberar` y `cambiarEstado` protegen el ciclo de una mesa.

Pedido

Clase abstracta con fecha, estado, total, observaciones y detalles. `agregarDetalle` y `calcularTotal` mantienen la cuenta.

PedidoLocal

Hereda de Pedido y agrega la Mesa. Es el tipo concreto usado por el restaurante.

EstadoPedido

Limita el proceso a pendiente, en preparación, listo, entregado o cancelado.

DetallePedido

Una línea del pedido. Valida cantidad/precio y `calcularSubtotal` multiplica ambos.

Pago

Relaciona pedido y caja, y conserva monto, cambio y fecha.

Caja / EstadoCaja

Representa una jornada de efectivo abierta o cerrada con montos inicial y final.

Servicios

SesionService

Inicia/cierra sesión, resuelve el empleado actual, comprueba roles y administra empleados.

CatalogoService

Lista y guarda categorías/productos. Valida precio, categoría y relaciones; compara categorías por id.

AtencionService

Lista, crea, edita y elimina mesas aplicando reglas de ocupación e historial.

PedidoService

Crea pedidos y controla los cambios pendiente → preparación → listo → entregado/cancelado.

FinanzasService

Abre/cierra caja, calcula saldos, registra pagos, calcula cambio y arma comprobantes.

DatosRestaurante

Contrato abstracto para listas, guardado, eliminación, recarga y transacciones.

DatosMySQL

Implementación real: delega en DAO, detecta cambios simultáneos y reconstruye relaciones entre objetos.

Validacion

Funciones pequeñas para textos, números y dinero. Centraliza mensajes y límites.

Claves

Protege y verifica contraseñas con PBKDF2, sal aleatoria y comparación segura.

Datos y DAO

ConexionDB

Abre conexiones JDBC y transacciones. `commit` guarda todo; `rollback` deshace todo ante error.

EmpleadoDAO

CRUD de empleado y búsqueda por usuario. `escribir` coloca parámetros y `leer` arma el objeto.

CategoriaDAO

CRUD de categoría y conversión de filas con `leerCategoria`.

ProductoDAO

CRUD de producto; hace JOIN con categoría y devuelve objetos relacionados.

MesaDAO

CRUD de mesa con lectura/escritura común de sus campos.

PedidoDAO

Guarda pedido y detalles juntos, busca/lista, cambia estado/total y agrega productos a una cuenta activa.

DetallePedidoDAO

Lee o inserta las líneas de un pedido dentro de la conexión que le presta PedidoDAO.

PagoDAO

Lista y registra pagos asociados a pedido y caja.

CajaDAO

CRUD de jornadas de caja.

ControlCambiosDAO

Lee o incrementa una revisión compartida para detectar ediciones simultáneas.

03_categoria_producto · src/main/java/com/garagecoffee/service/CatalogoService.java

```
                                    Categoria categoria, boolean disponible) {
        return datos.transaccion(() -> {
            sesion.exigir();
            String nombreValido = Validacion.texto(nombre, "Nombre", 150);
            String descripcionValida = Validacion.opcional(descripcion, "Descripción", 255);
            String imagenValida = Validacion.opcional(imagen, "Imagen", 255);
            BigDecimal precioValido = Validacion.dinero(precio, "Precio", true);
            Categoria categoriaValida = categoria == null ? null : datos.categorias.stream()
                    .filter(c -> c.getId() == categoria.getId()).findFirst().orElse(null);
            Validacion.exigir(categoriaValida != null, "Selecciona una categoría.");
            Validacion.exigir(!disponible || categoriaValida.isActivo(),
                    "Activa la categoría antes de ofrecer el producto.");
            Producto producto = id == 0 ? new Producto() : datos.productos.stream()
                    .filter(p -> p.getId() == id).findFirst().orElseThrow();
            if (id == 0) producto.setId(datos.siguienteId(Producto.class));
            producto.setNombre(nombreValido);
            producto.setDescripcion(descripcionValida);
            producto.setImagenUrl(imagenValida);
            producto.setPrecio(precioValido);
            producto.setCategoria(categoriaValida);
            producto.setDisponible(disponible);
            datos.guardar(producto);
            if (id == 0) datos.productos.add(producto);
```

Figura 4. Validación real: la categoría se resuelve por id antes de guardar el producto.

4. La base de datos y JDBC

La base es relacional: cada tabla guarda un tipo de ficha y los ids conectan una ficha con otra. El script DDL no está en el repositorio actual; la siguiente estructura está comprobada a partir de las consultas SQL de las DAO.

Tabla	Campos que usa el código	Relación
empleado	id, nombre, apellido, teléfono, email, usuario, password, rol, activo	Inicio de sesión
categoria	id, nombre, tipo, activo	1 → muchos producto
producto	id, categoria_id, nombre, descripción, imagen, precio, disponible	Pertenece a categoría
mesa	id, número, capacidad, estado	1 → muchos pedido
pedido	id, mesa_id, fecha, estado, total, observaciones	Pertenece a mesa
detalle_pedido	id, pedido_id, producto_id, cantidad, precio, subtotal	Une pedido/producto
caja	id, apertura, cierre, inicial, final, estado	1 → muchos pago

Tabla	Campos que usa el código	Relación
pago	id, pedido_id, caja_id, monto, cambio, fecha	Une pedido/caja
control_cambios	id, revisión	Sincronización

Por qué detalle_pedido guarda el precio

El producto puede costar Q15 hoy y Q18 mañana. El detalle conserva Q15 para que un pedido antiguo no cambie mágicamente. Eso se llama conservar el historial.

Transacciones

Una transacción es como una caja cerrada: o se guardan todos los cambios, o no se guarda ninguno. Al crear un pedido deben guardarse pedido, detalles y mesa ocupada. Si falla un detalle, rollback devuelve todo al estado anterior.

04_transaccion · src/main/java/com/garagecoffee/database/ConexionDB.java

```
41      public static void revertir(Connection conexion) throws SQLException { if (actual.get() != conexion) conexion.rollback(); }
42      public static boolean enTransaccion() { return actual.get() != null; }
43      @FunctionalInterface public interface Operacion<T> { T ejecutar() throws SQLException; }
44      public static <T> T transaccion(Operacion<T> operacion) throws SQLException {
45          if (enTransaccion()) return operacion.ejecutar();
46          try (Connection conexion = getConnection()) {
47              conexion.setAutoCommit(false);
48              actual.set(conexion);
49              try {
50                  T resultado = operacion.ejecutar();
51                  conexion.commit();
52                  return resultado;
53              } catch (SQLException | RuntimeException | Error e) {
54                  try { conexion.rollback(); } catch (SQLException rollback) { e.addSuppressed(rollback); }
55                  throw e;
56              } finally { actual.remove(); }
57          }
58      }
59      public static void closeConnection(Connection conexion) {
60          if (conexion != null && conexion != actual.get()) {
61              try { conexion.close(); } catch (SQLException e) { System.err.println("No se pudo cerrar la conexión."); }
62          }
```

Figura 5. ConexionDB confirma con commit o deshace con rollback.

DAO y SQL preparado

Cada DAO conoce una tabla. Los signos ? del PreparedStatement reciben valores por separado. Así los datos escritos por una persona no se convierten en parte del comando SQL.

05_dao_sql · src/main/java/com/garagecoffee/dao/ProductoDAO.java

```
            }
        }
    }

    // asigna el id generado.
    public boolean guardar(Producto producto) throws SQLException {
        String sql = "INSERT INTO producto "
                + "(categoria_id, nombre, descripcion, imagen_url, precio, disponible) "
                + "VALUES (?, ?, ?, ?, ?, ?)";

        try (ConexionDB.Recurso recurso = ConexionDB.abrir();
             PreparedStatement consulta = recurso.conexion.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            consulta.setInt(1, producto.getcategoria().getId());
            consulta.setString(2, producto.getNombre());
            consulta.setString(3, producto.getDescripcion());
            consulta.setString(4, producto.getImagenUrl());
            consulta.setBigDecimal(5, producto.getPrecio());
            consulta.setBoolean(6, producto.isDisponible());

            if (consulta.executeUpdate() == 0) {
                return false;
            }
            try (ResultSet claves = consulta.getGeneratedKeys()) {
                if (!claves.next()) {
                    throw new SQLException("No se obtuvo el ID del producto guardado");
                }
```

Figura 6. ProductoDAO inserta valores con parámetros y recupera el id generado.

5. Swing: la aplicación de escritorio

Swing usa JFrame para ventanas, JPanel para secciones y JDialog para formularios modales. Los archivos `.form` guardan el diseño editable de NetBeans; `initComponents()` es el Java generado por ese diseñador.

LoginFrame

JFrame de acceso. `ingresar` autentica, limpia la clave y abre DashboardFrame; `initComponents` dibuja la ventana.

DashboardFrame

JFrame principal. `enlace` conecta botones, `mostrar` abre/reutiliza módulos y `organizarMenu` acomoda el menú según el rol.

PanelModulo

JPanel base. `registrar` guarda listados y `recargar` actualiza todos.

PanelListado<T>

Tabla genérica con búsqueda y botones. `configurar`, `boton`, `accion`, `seleccionado`, `mensajeVacio` y `recargar` evitan repetir código.

Formulario

JPanel reutilizable que registra campos, los rellena, convierte texto a entero/decimal/fecha y ejecuta `guardar`.

InicioPanel

Resumen de mesas, pedidos y caja; `recargar` vuelve a calcular sus tarjetas.

EmpleadosPanel

Lista empleados y `editar` abre EmpleadoForm para crear o modificar.

CatalogoPanel

Lista productos/categorías. `producto` y `categoria` abren formularios y llaman CatalogoService.

AtencionPanel

Lista mesas, permite administrarlas y abre PedidoDialog para una mesa libre.

PedidosPanel

Lista pedidos; permite crear, entregar, cancelar y ver `detalle`.

CocinaPanel

Cola ordenada de cocina con acciones preparar, listo y detalle.

CajaPanel

Muestra cuentas/cajas. `cobrar` calcula cambio mientras se escribe y `recargar` resume la caja actual.

PedidoDialog

JDialog modal que reúne mesa, observaciones y carrito; agrega/cambia/quita líneas y envía a cocina.

TextoDialog

JDialog para ver comprobantes o comandas de texto.

EmpleadoForm

Campos de empleado, rol, clave y estado.

CategoriaForm

Campos de nombre, tipo y estado de categoría.

ProductoForm

Campos de producto, precio, categoría, imagen y disponibilidad.

MesaForm

Campos de número, capacidad y servicio.

AperturaCajaForm

Captura el efectivo inicial.

CierreCajaForm

Muestra lo esperado y captura lo contado.

CobroForm

Muestra total, efectivo y cambio.

CantidadForm

Captura una cantidad.

LineaPedidoForm

Selecciona producto y cantidad.

PedidoDatosForm

Selecciona mesa y observaciones.

Ui

Clase utilitaria para estilos, formatos, mensajes, impresión y exportación de texto.

06_swing · src/main/java/com/garagecoffee/view/CatalogoPanel.java

```
            registrar(categorias);
        }
        private void producto(Producto p) {
            Formulario f = new ProductoForm();
            var nombre = f.texto("Nombre", p == null ? "" : p.getNombre());
            var descripcion = f.texto("Descripción", p == null ? "" : p.getDescripcion());
            var imagen = f.texto("Referencia de imagen (opcional)", p == null ? "" : p.getImagenUrl());
            var precio = f.texto("Precio Q", p == null ? "" : p.getPrecio());
            var categoriasDisponibles = servicio.categorias();
            Categoria categoriaActual = p == null || p.getcategoria() == null ? null : categoriasDisponibles.stream()
                    .filter(c -> c.getId() == p.getcategoria().getId()).findFirst().orElse(null);
            var categoria = f.combo("Categoría", categoriasDisponibles, categoriaActual);
            var activo = f.check("Disponible", p == null || p.isDisponible());
            f.guardar(this, "Producto", () -> { servicio.guardarProducto(p == null ? 0 : p.getId(), f.valor(nombre), f.valor(descripcion),
                    f.valor(imagen), f.decimal(precio), f.elegido(categoria), activo.isSelected()); recargar(); });
        }
        private void categoria(Categoria c) {
            Formulario f = new CategoriaForm();
            var nombre = f.texto("Nombre", c == null ? "" : c.getNombre());
            var tipo = f.combo("Tipo", List.of(TipoCategoria.values()), c == null ? TipoCategoria.COMIDA : c.getTipo());
            var activo = f.check("Activa", c == null || c.isActivo());
            f.guardar(this, "Categoría", () -> { servicio.guardarCategoria(c == null ? 0 : c.getId(), f.valor(nombre), f.elegido(tipo), activo.isSelected()); recargar(); });
```

Figura 7. CatalogoPanel prepara campos Swing y delega el guardado a CatalogoService.

Por qué la interfaz no debe contener SQL

El botón solo recoge valores. CatalogoService valida y ProductoDAO guarda. Si mañana cambia la ventana, la regla sigue en un solo lugar. Esa separación hace más fácil entender y mantener el programa.

6. Spring: el puente para Android

Spring corre separado de Swing. Escucha en `127.0.0.1:8080`, recibe JSON, identifica al empleado, revisa el rol y delega en servicios/DAO. No abre ventanas.

BackendApplication

`main` inicia Spring Boot.

ConfiguracionDatos

Conecta el DataSource con ConexionDB y publica las DAO como beans para inyección.

ConsultaRestauranteService

`mesas` y `catalogo` leen y convierten modelos a records de respuesta.

OperacionRestauranteService

Consulta pedido activo y realiza crear, ampliar, entregar, cobrar y cambiar mesa dentro de transacciones.

RestauranteApi

Controlador REST bajo `/api`; cada método recibe una ruta y delega la regla al servicio.

SeguridadApi

Carga empleados, adapta la protección de claves y define qué rol puede usar cada ruta.

ErroresApi

Transforma errores de datos, conflictos y SQL en respuestas 400, 409 y 503 seguras.

Objetos que entran y salen de la API

Estos `record` son sobres pequeños para mover datos. Los de entrada reciben JSON de Android; los de respuesta devuelven únicamente lo que la app necesita.

Record	Explicación sencilla
MesaRespuesta	Devuelve id, número, capacidad y estado de una mesa.

Record	Explicación sencilla
ProductoRespuesta	Devuelve los datos visibles de un producto del menú.
DetallePedidoEntrada	Recibe un producto y la cantidad solicitada.
PedidoEntrada	Recibe la mesa, observaciones y la primera lista de productos.
AgregarPedidoEntrada	Recibe más productos para una cuenta que continúa abierta.
EstadoMesaEntrada	Recibe el nuevo estado que se quiere aplicar a una mesa.
PedidoRespuesta	Devuelve el identificador y el estado de un pedido recién creado.
DetallePedidoRespuesta	Devuelve una línea del pedido con nombre, cantidad, precio y subtotal.
PedidoMesaRespuesta	Devuelve la cuenta activa completa de una mesa.
CobroEntrada	Recibe el método de pago y el efectivo entregado.
CobroRespuesta	Devuelve total, efectivo, cambio y confirmación del cobro.

Rutas disponibles

Ruta	Qué hace	Roles
GET /api/estado	Comprueba que Spring responde	Público
GET /api/sesion	Devuelve usuario y roles	Autenticado
GET /api/mesas	Lista mesas	Admin, mesero, cajero
GET /api/catalogo	Lista productos disponibles	Admin, mesero, cajero, cocina
GET /api/mesas/{id}/pedido-activo	Cuenta abierta de la mesa	Admin, mesero, cajero
POST /api/pedidos	Crea pedido	Admin, mesero, cajero
PUT /api/pedidos/{id}/productos	Agrega productos	Admin, mesero, cajero
PUT /api/pedidos/{id}/entregar	Marca entregado	Admin, mesero, cajero
POST /api/pedidos/{id}/cobrar	Registra pago y libera mesa	Admin, mesero, cajero
PUT /api/mesas/{id}/estado	Libre/fuera de servicio	Admin, mesero

07_spring_api · backend/src/main/java/com/garagecoffee/api/RestauranteApi.java

```
25      @GetMapping("/catalogo") public List<ConsultaRestauranteService.ProductoRespuesta> catalogo() throws SQLException { return consultas.catalogo(); }
26      @GetMapping("/mesas/{id}/pedido-activo")
27      public OperacionRestauranteService.PedidoMesaRespuesta pedidoActivo(@PathVariable int id) throws SQLException {
28          return operaciones.pedidoActivo(id);
29      }
30      @PostMapping("/pedidos")
31      @ResponseStatus(org.springframework.http.HttpStatus.CREATED)
32      public OperacionRestauranteService.PedidoRespuesta crearPedido(
33              @RequestBody OperacionRestauranteService.PedidoEntrada pedido) throws SQLException {
34          return operaciones.crearPedido(pedido);
35      }
36      @PutMapping("/mesas/{id}/estado")
37      public ConsultaRestauranteService.MesaRespuesta cambiarEstado(
38              @PathVariable int id, @RequestBody OperacionRestauranteService.EstadoMesaEntrada estado) throws SQLException {
39          return operaciones.cambiarEstadoMesa(id, estado);
40      }
41      @PutMapping("/pedidos/{id}/entregar")
42      public OperacionRestauranteService.PedidoMesaRespuesta entregar(@PathVariable int id) throws SQLException {
43          return operaciones.entregarPedido(id);
44      }
45      @PutMapping("/pedidos/{id}/productos")
46      public OperacionRestauranteService.PedidoMesaRespuesta agregarProductos(
47              @PathVariable int id, @RequestBody OperacionRestauranteService.AgregarPedidoEntrada entrada)
48              throws SQLException {
49          return operaciones.agregarAlPedido(id, entrada);
50      }
51      @PostMapping("/pedidos/{id}/cobrar")
```

Figura 8. RestauranteApi traduce rutas HTTP en llamadas al servicio.

Seguridad

HTTP Basic manda usuario y contraseña en cada solicitud. Base64 no es cifrado; por eso el servidor está limitado al equipo local y una instalación real necesitaría HTTPS. La clave almacenada se verifica con PBKDF2 y nunca se devuelve en JSON.

08_seguridad · backend/src/main/java/com/garagecoffee/api/SeguridadApi.java

```
            }
        };
    }
    @Bean SecurityFilterChain permisos(HttpSecurity http) throws Exception {
        return http
                .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .requestCache(c -> c.disable())
                // valida las credenciales de cada solicitud.
                .csrf(c -> c.disable())
                .authorizeHttpRequests(a -> a
                        .requestMatchers(HttpMethod.GET, "/api/estado").permitAll()
                        .requestMatchers(HttpMethod.GET, "/api/sesion").authenticated()
                        .requestMatchers(HttpMethod.GET, "/api/mesas").hasAnyRole("ADMINISTRADOR", "MESERO", "CAJERO")
                        .requestMatchers(HttpMethod.GET, "/api/mesas/*/pedido-activo").hasAnyRole("ADMINISTRADOR", "MESERO", "CAJERO")
                        .requestMatchers(HttpMethod.GET, "/api/catalogo").hasAnyRole("ADMINISTRADOR", "MESERO", "CAJERO", "COCINA")
                        .requestMatchers(HttpMethod.POST, "/api/pedidos").hasAnyRole("ADMINISTRADOR", "MESERO", "CAJERO")
                        .requestMatchers(HttpMethod.POST, "/api/pedidos/*/cobrar").hasAnyRole("ADMINISTRADOR", "MESERO", "CAJERO")
                        .requestMatchers(HttpMethod.PUT, "/api/mesas/*/estado").hasAnyRole("ADMINISTRADOR", "MESERO")
                        .requestMatchers(HttpMethod.PUT, "/api/pedidos/*/entregar").hasAnyRole("ADMINISTRADOR", "MESERO", "CAJERO")
                        .requestMatchers(HttpMethod.PUT, "/api/pedidos/*/productos").hasAnyRole("ADMINISTRADOR", "MESERO", "CAJERO")
                        .anyRequest().denyAll())
                .httpBasic(Customizer.withDefaults())
                .exceptionHandling(e -> e
                        .authenticationEntryPoint((req, res, ex) -> {
                            res.setStatus(401); res.setContentType("application/json");
```

Figura 9. Spring Security declara roles por ruta y rechaza lo que no está permitido.

7. Android, Kotlin y Jetpack Compose

Android mezcla Kotlin para actividades/pantallas y Java para el cliente HTTP. Compose dibuja la interfaz con funciones anotadas `@Composable`; no usa XML para estas pantallas.

SesionApp

Application que mantiene una sola instancia de ApiCliente mientras vive el proceso.

ApiCliente

Cliente HTTP Java. Guarda autorización en memoria, forma JSON, ejecuta fuera del hilo visual y traduce errores.

AccesoActivity

Estado del login. `onCreate` monta Compose e `ingresar` llama Spring y abre InicioActivity.

InicioActivity

Coordina mesas, menú y pedido. Convierte JSON, refresca cada 4 segundos, crea/amplía/entrega/cobra y vuelve al acceso.

PantallaAcceso

Composable del login; recibe estado y callbacks, no conoce HTTP.

PantallaInicio

Contiene data classes y composables de mesas, catálogo, pedido, entrega y cobro.

TemaGarageCoffee

Colores y `GarageCoffeeTheme`, que envuelve todas las pantallas con Material 3.

Datos que Compose muestra en pantalla

Tipo	Explicación sencilla
MesaUi	Representa una tarjeta de mesa dentro de la app.
ProductoUi	Representa un producto, su precio y la cantidad elegida.
DetallePedidoUi	Representa una línea ya guardada en la cuenta.
PedidoMesaUi	Agrupa el pedido activo, sus detalles, estado y total.
Seccion	Enum que indica si se muestran mesas, pedido u otra sección de la pantalla.

Estado y recomposición

`mutableStateOf` es como una libreta que Compose vigila. Cuando cambia `pedidoActivo`, Compose vuelve a ejecutar la parte visual necesaria. `remember` conserva valores temporales, como cantidades o efectivo, mientras la pantalla siga siendo la misma.

10_estado_android · android/app/src/main/java/garage_coffee/app/InicioActivity.kt

```
28      private var productos by mutableStateOf<List<ProductoUi>>(emptyList())
29      private var mesaSeleccionada by mutableStateOf<MesaUi?>(null)
30      private var pedidoActivo by mutableStateOf<PedidoMesaUi?>(null)
31      private var mensaje by mutableStateOf("")
32      private var cargando by mutableStateOf(false)
33      private var consultaPedidoEnCurso = false
34      private val actualizador = Handler(Looper.getMainLooper())
35      private val refrescarPedido = object : Runnable {
36          override fun run() {
37              if (mesaSeleccionada == null || isFinishing || isDestroyed) return
38              cargarPedidoActivo(true)
39              actualizador.postDelayed(this, 4000)
40          }
41      }
42
43      override fun onCreate(savedInstanceState: Bundle?) {
44          super.onCreate(savedInstanceState)
45          sesion = application as SesionApp
46          if (!sesion.api().iniciada()) {
47              volverAlAcceso()
48              return
49          }
50
51          val rol = sesion.api().rol().orEmpty()
52          val puedeVerMesas = rol != "COCINA" && rol != "REPARTIDOR"
53          val puedeVerCatalogo = rol != "REPARTIDOR"
```

Figura 10. InicioActivity conserva estado y consulta el pedido cada cuatro segundos.

Cliente HTTP

ApiCliente trabaja en un ExecutorService para no congelar la pantalla. Cuando termina, usa Handler para regresar al hilo visual y llama `correcto` o `error`.

09_cliente_http · android/app/src/main/java/garage_coffee/app/ApiCliente.java

```java
    public void catalogo(Respuesta<JSONArray> respuesta) {
        tarea(() -> new JSONArray(pedir("/api/catalogo", autorizacion)), respuesta);
    }
    public void crearPedido(int mesaId, String observaciones, Map<Integer, Integer> cantidades,
            Respuesta<JSONObject> respuesta) {
        tarea(() -> {
            JSONObject pedido = new JSONObject();
            pedido.put("mesaId", mesaId);
            pedido.put("observaciones", observaciones == null ? "" : observaciones);
            JSONArray detalles = new JSONArray();
            for (var entrada : cantidades.entrySet()) {
                JSONObject detalle = new JSONObject();
                detalle.put("productoId", entrada.getKey());
                detalle.put("cantidad", entrada.getValue());
                detalles.put(detalle);
            }
            pedido.put("detalles", detalles);
            return new JSONObject(solicitar("/api/pedidos", "POST", pedido.toString(), autorizacion));
        }, respuesta);
    }
    public void cambiarEstadoMesa(int mesaId, String estado, Respuesta<JSONObject> respuesta) {
        tarea(() -> {
            JSONObject cambio = new JSONObject();
            cambio.put("estado", estado);
            return new JSONObject(solicitar(
                    "/api/mesas/" + mesaId + "/estado", "PUT", cambio.toString(), autorizacion));
```

Figura 11. Android forma el JSON del pedido y lo envía a Spring.

Pantallas Compose

11_compose · android/app/src/main/java/garage_coffee/app/ui/PantallaInicio.kt

```kotlin
        }
    }
}
@Composable
fun PantallaPedidoMesa(
    mesa: MesaUi,
    productos: List<ProductoUi>,
    pedido: PedidoMesaUi?,
    mensaje: String,
    cargando: Boolean,
    puedeCambiarEstado: Boolean,
    alVolver: () -> Unit,
    alActualizarPedido: () -> Unit,
    alCambiarEstado: (String) -> Unit,
    alEntregar: (Int) -> Unit,
    alCobrar: (Int, String) -> Unit,
    alEnviar: (String, Map<Int, Int>) -> Unit
) {
    val edicion = "${mesa.id}:${pedido?.id ?: 0}:${pedido?.total.orEmpty()}"
    val cantidades = remember(edicion) { mutableStateMapOf<Int, Int>() }
    var observaciones by remember(edicion) { mutableStateOf("") }
    val total = productos.fold(BigDecimal.ZERO) { suma, producto ->
        suma + (producto.precio.toBigDecimalOrNull() ?: BigDecimal.ZERO)
            .multiply(BigDecimal.valueOf((cantidades[producto.id] ?: 0).toLong()))
    }.setScale(2, RoundingMode.HALF_UP)
    val permitePedido = mesa.estado == "LIBRE" || pedido?.activo == true

    Column(
```

Figura 12. Un composable recibe estado y callbacks; la lógica HTTP queda fuera de la pantalla.

8. Flujos completos del restaurante

Pedido de mesa



Cancelar solo es posible mientras está pendiente; la mesa queda libre al entregar y completar el pago.

- El mesero abre una mesa libre y elige productos.
- PedidoService u OperacionRestauranteService valida y ocupa la mesa.
- Cocina cambia pendiente → en preparación → listo desde Swing.
- Android refresca el pedido y muestra que está listo.
- El mesero marca entregado; todavía puede verse el total pendiente.
- Con caja abierta, el cobro registra Pago, calcula cambio y libera la mesa.

Agregar más productos

Mientras la mesa siga ocupada y el pedido no esté cerrado, Android puede llamar `agregarAlPedido`. El servidor valida productos, guarda nuevas líneas, recalcula total y vuelve el estado a pendiente para que cocina vea la ampliación.

Cambios simultáneos

Swing mantiene listas en memoria. Spring puede cambiar MySQL al mismo tiempo. `control_cambios` aumenta una revisión en cada escritura; Swing compara esa revisión antes de guardar. Si cambió, pide actualizar para no sobrescribir trabajo de otra persona.

9. Configuración, compilación y ejecución

Parte	Necesita	Cómo se inicia
Swing	JDK 25 + MySQL	Raíz Maven / Proyecto2Progra2
Spring	JDK 25 + MySQL	backend/mvnw spring-boot:run
Android	Android Studio, SDK 37, emulador API 34+	Abrir android/ y ejecutar app

- `pom.xml`: compila Swing y agrega MySQL Connector.
- `backend/pom.xml`: agrega Spring Web, JDBC y Security; depende del proyecto principal.
- `application.properties`: puerto, DataSource y carga opcional de `garage-db.properties`.
- `garage-db.properties`: credenciales locales; está excluido de Git.
- `android/app/build.gradle.kts`: Compose, SDK y `API_BASE_URL`.
- `AndroidManifest.xml`: Internet, SesionApp y actividades.
- `strings.xml`: textos visibles de Android.
- `mvnw` y `gradlew`: evitan instalar Maven/Gradle globalmente.

Dato importante: el repositorio actual no incluye el script SQL. Cada equipo necesita recibir o conservar por separado el esquema compatible con las nueve tablas descritas.

Comandos útiles

Objetivo	Comando
Compilar Swing	./backend/mvnw -f pom.xml clean package
Instalar biblioteca principal	cd backend && ./mvnw -f ../pom.xml install
Iniciar Spring	cd backend && ./mvnw spring-boot:run
Compilar Android	cd android && ./gradlew :app:assembleDebug
Conectar emulador a Spring	adb reverse tcp:8080 tcp:8080

10. Errores comunes explicados

No conecta MySQL

Revisar que MySQL esté iniciado, URL/puerto correctos, esquema existente y garage-db.properties local.

Android dice que el servidor tardó

Iniciar Spring y ejecutar adb reverse; 127.0.0.1 del emulador requiere ese puente.

401

Usuario/contraseña de empleado incorrectos o cuenta inactiva.

403

El usuario entró bien, pero su rol no puede usar esa ruta.

503

Spring funciona, pero no pudo consultar MySQL.

Categoría requerida al editar

La solución actual resuelve la categoría seleccionada por id, incluso si se recargaron objetos.

Otra sesión modificó los datos

Pulsar Actualizar y volver a abrir el formulario; es protección contra sobrescritura.

Gradle no encuentra proyecto

Abrir la carpeta android/, no la raíz del monorepo.

Apéndice A · Funciones declaradas en el código

Este índice se genera leyendo los archivos actuales. Sirve para localizar rápidamente una función. Los getters/setters repetidos significan simplemente leer/cambiar un atributo; los capítulos anteriores explican las reglas importantes.

AccesoActivity

android/app/src/main/java/garage_coffee/app/AccesoActivity.kt

Función	Explicación sencilla
onCreate(savedInstanceState: Bundle?)	crea la actividad y conecta estado con la pantalla
ingresar()	intenta iniciar sesión y comunica el resultado
correcto(valor: String)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla

ApiCliente

android/app/src/main/java/garage_coffee/app/ApiCliente.java

Función	Explicación sencilla
iniciada()	indica si Android conserva una sesión válida en memoria
usuario()	devuelve el nombre del empleado autenticado
rol()	devuelve el rol del empleado autenticado
salir()	olvida la sesión guardada en memoria
ingresar(String nombre, String clave, Respuesta<String> respuesta)	intenta iniciar sesión y comunica el resultado
mesas(Respuesta<JSONArray> respuesta)	obtiene o devuelve la lista de mesas permitida
catalogo(Respuesta<JSONArray> respuesta)	obtiene o devuelve los productos visibles del menú
crearPedido(int mesaId, String observaciones, Map<Integer, Integer> cantidades, Respuesta<JSONObject> respuesta)	crea una orden nueva para una mesa libre
cambiarEstadoMesa(int mesaId, String estado, Respuesta<JSONObject> respuesta)	aplica el cambio solicitado después de validarlo
agregarAlPedido(int pedidoId, String observaciones, Map<Integer, Integer> cantidades, Respuesta<JSONObject> respuesta)	añade productos a una orden que sigue abierta
pedidoActivo(int mesaId, Respuesta<JSONObject> respuesta)	busca la cuenta que todavía pertenece a una mesa
entregarPedido(int pedidoId, Respuesta<JSONObject> respuesta)	marca como entregada una orden que estaba lista
cobrarPedido(int pedidoId, String efectivo, Respuesta<JSONObject> respuesta)	registra el efectivo y cierra la mesa cuando corresponde
tarea(Operacion<T> operacion, Respuesta<T> respuesta)	ejecuta una llamada fuera del hilo visual y devuelve el resultado
pedir(String ruta, String credenciales)	realiza una consulta HTTP GET
solicitar(String ruta, String metodo, String cuerpo, String credenciales)	abre HTTP, agrega credenciales/cuerpo e interpreta la respuesta
leer(InputStream entrada)	convierte una fila SQL en un objeto Java

InicioActivity

android/app/src/main/java/garage_coffee/app/InicioActivity.kt

Función	Explicación sencilla
run()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
onCreate(savedInstanceState: Bundle?)	crea la actividad y conecta estado con la pantalla
mostrar(nueva: Seccion)	abre o actualiza la sección solicitada
abrirMesa(mesa: MesaUi)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
correcto(datos: JSONArray)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
cargarPedidoActivo(silencioso: Boolean)	carga datos necesarios para continuar
correcto(datos: JSONObject)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
actualizarMesaAbierta()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
correcto(datos: JSONArray)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
enviarPedido(observaciones: String, cantidades: Map<Int, Int>)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
correcto(datos: JSONObject)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
correcto(datos: JSONObject)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
cambiarEstado(nuevo: String)	aplica el cambio solicitado después de validarlo
correcto(datos: JSONObject)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
entregarPedido(id: Int)	marca como entregada una orden que estaba lista
correcto(datos: JSONObject)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
cobrarPedido(id: Int, efectivo: String)	registra el efectivo y cierra la mesa cuando corresponde
correcto(datos: JSONObject)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
respuesta(esperada: Seccion, recibir: (JSONArray) -> Unit)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
correcto(datos: JSONArray)	recibe el resultado exitoso de una llamada asíncrona
error(texto: String)	recibe un mensaje cuando la llamada falla
convertirMesas(datos: JSONArray)	transforma datos JSON en un objeto sencillo de pantalla
convertirMesa(mesa: JSONObject)	transforma datos JSON en un objeto sencillo de pantalla
convertirProductos(datos: JSONArray)	transforma datos JSON en un objeto sencillo de pantalla
convertirPedido(datos: JSONObject)	transforma datos JSON en un objeto sencillo de pantalla
corregirTexto(valor: String)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
detenerActualizacion()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
onDestroy()	detiene trabajos cuando Android destruye la actividad
volverAlAcceso()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

SesionApp

android/app/src/main/java/garage_coffee/app/SesionApp.java

Función	Explicación sencilla
api()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

PantallaAcceso

android/app/src/main/java/garage_coffee/app/ui/PantallaAcceso.kt

Función	Explicación sencilla
PantallaAcceso(usuario: String, clave: String, mensaje: String, cargando: Boolean, alCambiarUsuario: (String) ->...)	constructor: crea y prepara un objeto de esta clase

PantallaInicio

android/app/src/main/java/garage_coffee/app/ui/PantallaInicio.kt

Función	Explicación sencilla
PantallaInicio(usuario: String, rol: String, seccion: Seccion, puedeVerMesas: Boolean, puedeVerCatalogo: Boolean...)	constructor: crea y prepara un objeto de esta clase
PantallaPedidoMesa(mesa: MesaUi, productos: List<ProductoUi>, pedido: PedidoMesaUi?, mensaje: String, cargando: Bool...)	constructor: crea y prepara un objeto de esta clase
PedidoActivoTarjeta(pedido: PedidoMesaUi, cargando: Boolean, alActualizar: () -> Unit, alEntregar: (Int) -> Unit, alC...)	constructor: crea y prepara un objeto de esta clase
Cabecera(usuario: String, rol: String)	constructor: crea y prepara un objeto de esta clase
MensajeEstado(mensaje: String, cargando: Boolean)	constructor: crea y prepara un objeto de esta clase
BotonSeccion(texto: String, seleccionado: Boolean, habilitado: Boolean, modifier: Modifier, alPulsar: () -> Unit)	constructor: crea y prepara un objeto de esta clase
MesaTarjeta(mesa: MesaUi, alPulsar: () -> Unit)	constructor: crea y prepara un objeto de esta clase
ProductoTarjeta(producto: ProductoUi)	constructor: crea y prepara un objeto de esta clase
ProductoPedidoTarjeta(producto: ProductoUi, cantidad: Int, habilitado: Boolean, alCambiar: (Int) -> Unit)	constructor: crea y prepara un objeto de esta clase
ProductoDescripcion(producto: ProductoUi, modifier: Modifier = Modifier)	constructor: crea y prepara un objeto de esta clase

TemaGarageCoffee

android/app/src/main/java/garage_coffee/app/ui/TemaGarageCoffee.kt

Función	Explicación sencilla
GarageCoffeeTheme(contenido: @Composable () -> Unit)	constructor: crea y prepara un objeto de esta clase

BackendApplication

backend/src/main/java/com/garagecoffee/api/BackendApplication.java

Función	Explicación sencilla
main(String[] args)	punto de entrada que inicia esta parte del sistema

ConfiguracionDatos

backend/src/main/java/com/garagecoffee/api/ConfiguracionDatos.java

Función	Explicación sencilla
ConfiguracionDatos(DataSource fuente)	constructor: crea y prepara un objeto de esta clase
empleadoDAO()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
mesaDAO()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
productoDAO()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
pedidoDAO()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
pagoDAO()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
cajaDAO()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
controlCambiosDAO()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

ConsultaRestauranteService

backend/src/main/java/com/garagecoffee/api/ConsultaRestauranteService.java

Función	Explicación sencilla
ConsultaRestauranteService(MesaDAO mesas, ProductoDAO productos)	constructor: crea y prepara un objeto de esta clase
mesas()	obtiene o devuelve la lista de mesas permitida
catalogo()	obtiene o devuelve los productos visibles del menú

ErroresApi

backend/src/main/java/com/garagecoffee/api/ErroresApi.java

Función	Explicación sencilla
datosInvalidos(IllegalArgumentException error)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
conflicto(IllegalStateException error)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
baseNoDisponible(SQLException error)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

OperacionRestauranteService

backend/src/main/java/com/garagecoffee/api/OperacionRestauranteService.java

Función	Explicación sencilla
OperacionRestauranteService(MesaDAO mesas, ProductoDAO productos, PedidoDAO pedidos, PagoDAO pagos, CajaDAO cajas, ControlCambiosDAO cambios)	constructor: crea y prepara un objeto de esta clase
pedidoActivo(int mesaId)	busca la cuenta que todavía pertenece a una mesa
crearPedido(PedidoEntrada entrada)	crea una orden nueva para una mesa libre
cambiarEstadoMesa(int id, EstadoMesaEntrada entrada)	aplica el cambio solicitado después de validarlo
agregarAlPedido(int id, AgregarPedidoEntrada entrada)	añade productos a una orden que sigue abierta
entregarPedido(int id)	marca como entregada una orden que estaba lista
cobrarPedido(int id, CobroEntrada entrada)	registra el efectivo y cierra la mesa cuando corresponde
pagado(int pedidoId, List<Pago> pagosActuales)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
cantidadesValidas(List<DetallePedidoEntrada> detalles)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
respuesta(PedidoLocal pedido, List<Pago> pagosActuales, int mesaId)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

RestauranteApi

backend/src/main/java/com/garagecoffee/api/RestauranteApi.java

Función	Explicación sencilla
RestauranteApi(ConsultaRestauranteService consultas, OperacionRestauranteService operaciones)	constructor: crea y prepara un objeto de esta clase
pedidoActivo(@PathVariable int id)	busca la cuenta que todavía pertenece a una mesa
crearPedido(@RequestBody OperacionRestauranteService.PedidoEntrada pedido)	crea una orden nueva para una mesa libre
cambiarEstado(@PathVariable int id, @RequestBody OperacionRestauranteService.EstadoMesaEntrada estado)	aplica el cambio solicitado después de validarlo
entregar(@PathVariable int id)	marca un pedido listo como entregado
agregarProductos(@PathVariable int id, @RequestBody OperacionRestauranteService.AgregarPedidoEntrada entrada)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
cobrar(@PathVariable int id, @RequestBody OperacionRestauranteService.CobroEntrada cobro)	registra efectivo, pago y cambio

SeguridadApi

backend/src/main/java/com/garagecoffee/api/SeguridadApi.java

Función	Explicación sencilla
empleados(EmpleadoDAO dao)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
claves()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
PasswordEncoder()	constructor: crea y prepara un objeto de esta clase
encode(CharSequence clave)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
matches(CharSequence clave, String guardada)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
permisos(HttpSecurity http)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

Proyecto2Progra2

src/main/java/com/garagecoffee/Proyecto2Progra2.java

Función	Explicación sencilla
main(String[] args)	punto de entrada que inicia esta parte del sistema

RestauranteController

src/main/java/com/garagecoffee/controller/RestauranteController.java

Función	Explicación sencilla
RestauranteController()	constructor: crea y prepara un objeto de esta clase
RestauranteController(DatosRestaurante datos)	constructor: crea y prepara un objeto de esta clase
recargar()	vuelve a consultar datos y refresca la vista
sesion()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
catalogo()	obtiene o devuelve los productos visibles del menú
atencion()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
pedidos()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
finanzas()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

CajaDAO

src/main/java/com/garagecoffee/dao/CajaDAO.java

Función	Explicación sencilla
listar()	trae todos los registros de este tipo
listar(Connection conexion)	trae todos los registros de este tipo
buscarPorId(int id)	busca un registro por su identificador
buscarPorId(Connection conexion, int id)	busca un registro por su identificador
guardar(Caja caja)	crea o conserva cambios después de validar
actualizar(Caja caja)	modifica un registro existente
eliminar(int id)	borra un registro si sus relaciones lo permiten
escribir(PreparedStatement consulta, Caja caja)	coloca los atributos del objeto en un PreparedStatement
leer(Connection conexion, ResultSet resultado)	convierte una fila SQL en un objeto Java

CategoriaDAO

src/main/java/com/garagecoffee/dao/CategoriaDAO.java

Función	Explicación sencilla
listar()	trae todos los registros de este tipo
buscarPorId(int id)	busca un registro por su identificador
guardar(Categoria categoria)	crea o conserva cambios después de validar
actualizar(Categoria categoria)	modifica un registro existente
eliminar(int id)	borra un registro si sus relaciones lo permiten
leerCategoria(ResultSet resultado)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

ControlCambiosDAO

src/main/java/com/garagecoffee/dao/ControlCambiosDAO.java

Función	Explicación sencilla
revision(boolean bloquear)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
avanzar()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

DetallePedidoDAO

src/main/java/com/garagecoffee/dao/DetallePedidoDAO.java

Función	Explicación sencilla
listarPorPedido(Connection conexion, int pedidoId)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardarDetalles(Connection conexion, int pedidoId, List<DetallePedido> detalles)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

EmpleadoDAO

src/main/java/com/garagecoffee/dao/EmpleadoDAO.java

Función	Explicación sencilla
listar()	trae todos los registros de este tipo
listar(Connection conexion)	trae todos los registros de este tipo
buscarPorId(int id)	busca un registro por su identificador
buscarPorId(Connection conexion, int id)	busca un registro por su identificador
guardar(Empleado empleado)	crea o conserva cambios después de validar
buscarPorNombreUsuario(String nombreUsuario)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
actualizar(Empleado empleado)	modifica un registro existente
eliminar(int id)	borra un registro si sus relaciones lo permiten
escribir(PreparedStatement consulta, Empleado empleado)	coloca los atributos del objeto en un PreparedStatement
leer(Connection conexion, ResultSet resultado)	convierte una fila SQL en un objeto Java

MesaDAO

src/main/java/com/garagecoffee/dao/MesaDAO.java

Función	Explicación sencilla
listar()	trae todos los registros de este tipo
listar(Connection conexion)	trae todos los registros de este tipo
buscarPorId(int id)	busca un registro por su identificador
buscarPorId(Connection conexion, int id)	busca un registro por su identificador
guardar(Mesa mesa)	crea o conserva cambios después de validar
actualizar(Mesa mesa)	modifica un registro existente
eliminar(int id)	borra un registro si sus relaciones lo permiten
escribir(PreparedStatement consulta, Mesa mesa)	coloca los atributos del objeto en un PreparedStatement
leer(Connection conexion, ResultSet resultado)	convierte una fila SQL en un objeto Java

PagoDAO

src/main/java/com/garagecoffee/dao/PagoDAO.java

Función	Explicación sencilla
listar()	trae todos los registros de este tipo
guardar(Pago pago)	crea o conserva cambios después de validar

PedidoDAO

src/main/java/com/garagecoffee/dao/PedidoDAO.java

Función	Explicación sencilla
listar()	trae todos los registros de este tipo
buscarPorId(int id)	busca un registro por su identificador
buscarPorId(Connection conexion, int id)	busca un registro por su identificador
guardar(Pedido pedido)	crea o conserva cambios después de validar
actualizar(Pedido pedido)	modifica un registro existente

Función	Explicación sencilla
agregarDetalles(Pedido pedido, List<DetallePedido> nuevos)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
leer(Connection conexion, ResultSet fila)	convierte una fila SQL en un objeto Java

ProductoDAO

src/main/java/com/garagecoffee/dao/ProductoDAO.java

Función	Explicación sencilla
listar()	trae todos los registros de este tipo
buscarPorId(int id)	busca un registro por su identificador
buscarPorId(Connection conexion, int id)	busca un registro por su identificador
guardar(Producto producto)	crea o conserva cambios después de validar
actualizar(Producto producto)	modifica un registro existente
eliminar(int id)	borra un registro si sus relaciones lo permiten
leerProducto(ResultSet resultado)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

ConexionDB

src/main/java/com/garagecoffee/database/ConexionDB.java

Función	Explicación sencilla
usarDataSource(javax.sql.DataSource dataSource)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
configuracion(String clave, String defecto)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
getConnection()	lee el atributo indicado sin modificarlo
abrir()	abre el recurso o la caja correspondiente
Recurso(Connection conexion, boolean propia)	constructor: crea y prepara un objeto de esta clase
close()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
confirmar(Connection conexion)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
revertir(Connection conexion)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
enTransaccion()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
transaccion(Operacion<T> operacion)	ejecuta varios cambios como una sola unidad
closeConnection(Connection conexion)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

Caja

src/main/java/com/garagecoffee/model/Caja.java

Función	Explicación sencilla
Caja()	constructor: crea y prepara un objeto de esta clase
Caja(int id, LocalDateTime fechaApertura, LocalDateTime fechaCierre, BigDecimal montoInicial, BigDecimal montoFinal, EstadoCaja estado)	constructor: crea y prepara un objeto de esta clase
getId()	lee el atributo indicado sin modificarlo
setId(int id)	cambia el atributo indicado
getFechaApertura()	lee el atributo indicado sin modificarlo

Función	Explicación sencilla
setFechaApertura(LocalDateTime fechaApertura)	cambia el atributo indicado
getFechaCierre()	lee el atributo indicado sin modificarlo
setFechaCierre(LocalDateTime fechaCierre)	cambia el atributo indicado
getMontoInicial()	lee el atributo indicado sin modificarlo
setMontoInicial(BigDecimal montoInicial)	cambia el atributo indicado
getMontoFinal()	lee el atributo indicado sin modificarlo
setMontoFinal(BigDecimal montoFinal)	cambia el atributo indicado
getEstado()	lee el atributo indicado sin modificarlo
setEstado(EstadoCaja estado)	cambia el atributo indicado

Categoria

src/main/java/com/garagecoffee/model/Categoria.java

Función	Explicación sencilla
Categoria()	constructor: crea y prepara un objeto de esta clase
Categoria(int id, String nombre, TipoCategoria tipo, boolean activo)	constructor: crea y prepara un objeto de esta clase
getId()	lee el atributo indicado sin modificarlo
getNombre()	lee el atributo indicado sin modificarlo
getTipo()	lee el atributo indicado sin modificarlo
isActivo()	responde verdadero o falso sobre ese estado
setId(int id)	cambia el atributo indicado
setNombre(String nombre)	cambia el atributo indicado
setTipo(TipoCategoria tipo)	cambia el atributo indicado
setActivo(boolean activo)	cambia el atributo indicado

DetallePedido

src/main/java/com/garagecoffee/model/DetallePedido.java

Función	Explicación sencilla
DetallePedido()	constructor: crea y prepara un objeto de esta clase
DetallePedido(int id, int cantidad, BigDecimal precioUnitario, Producto producto)	constructor: crea y prepara un objeto de esta clase
calcularSubtotal()	multiplica cantidad por precio unitario
getId()	lee el atributo indicado sin modificarlo
setId(int id)	cambia el atributo indicado
getPedidoId()	lee el atributo indicado sin modificarlo
setPedidoId(int pedidoId)	cambia el atributo indicado
getCantidad()	lee el atributo indicado sin modificarlo
setCantidad(int cantidad)	cambia el atributo indicado
getPrecioUnitario()	lee el atributo indicado sin modificarlo

Función	Explicación sencilla
setPrecioUnitario(BigDecimal precioUnitario)	cambia el atributo indicado
getSubtotal()	lee el atributo indicado sin modificarlo
setSubtotal(BigDecimal subtotal)	cambia el atributo indicado
getProducto()	lee el atributo indicado sin modificarlo
setProducto(Producto producto)	cambia el atributo indicado
validarCantidad(int cantidad)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
validarPrecioUnitario(BigDecimal precioUnitario)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

Empleado

src/main/java/com/garagecoffee/model/Empleado.java

Función	Explicación sencilla
Empleado()	constructor: crea y prepara un objeto de esta clase
Empleado(int id, String nombre, String apellido, String telefono, String email, String nombreUsuario, String password, RolEmpleado rol, boolean activo)	constructor: crea y prepara un objeto de esta clase
getId()	lee el atributo indicado sin modificarlo
setId(int id)	cambia el atributo indicado
getNombre()	lee el atributo indicado sin modificarlo
setNombre(String nombre)	cambia el atributo indicado
getApellido()	lee el atributo indicado sin modificarlo
setApellido(String apellido)	cambia el atributo indicado
getTelefono()	lee el atributo indicado sin modificarlo
setTelefono(String telefono)	cambia el atributo indicado
getEmail()	lee el atributo indicado sin modificarlo
setEmail(String email)	cambia el atributo indicado
getNombreUsuario()	lee el atributo indicado sin modificarlo
setNombreUsuario(String nombreUsuario)	cambia el atributo indicado
getPassword()	lee el atributo indicado sin modificarlo
setPassword(String password)	cambia el atributo indicado
getRol()	lee el atributo indicado sin modificarlo
setRol(RolEmpleado rol)	cambia el atributo indicado
isActivo()	responde verdadero o falso sobre ese estado
setActivo(boolean activo)	cambia el atributo indicado

Mesa

src/main/java/com/garagecoffee/model/Mesa.java

Función	Explicación sencilla
Mesa()	constructor: crea y prepara un objeto de esta clase
Mesa(int id, int numero, int capacidad, EstadoMesa estado)	constructor: crea y prepara un objeto de esta clase

Función	Explicación sencilla
ocupar()	marca una mesa como ocupada aplicando validaciones
liberar()	marca una mesa como libre
cambiarEstado(EstadoMesa nuevoEstado)	aplica el cambio solicitado después de validarlo
getId()	lee el atributo indicado sin modificarlo
setId(int id)	cambia el atributo indicado
getNumero()	lee el atributo indicado sin modificarlo
setNumero(int numero)	cambia el atributo indicado
getCapacidad()	lee el atributo indicado sin modificarlo
setCapacidad(int capacidad)	cambia el atributo indicado
getEstado()	lee el atributo indicado sin modificarlo
setEstado(EstadoMesa estado)	cambia el atributo indicado
validarCapacidad(int capacidad)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

Pago

src/main/java/com/garagecoffee/model/Pago.java

Función	Explicación sencilla
Pago()	constructor: crea y prepara un objeto de esta clase
Pago(int id, Pedido pedido, Caja caja, BigDecimal monto, BigDecimal cambio, LocalDateTime fechaHora)	constructor: crea y prepara un objeto de esta clase
getId()	lee el atributo indicado sin modificarlo
setId(int id)	cambia el atributo indicado
getPedido()	lee el atributo indicado sin modificarlo
setPedido(Pedido pedido)	cambia el atributo indicado
getCaja()	lee el atributo indicado sin modificarlo
setCaja(Caja caja)	cambia el atributo indicado
getMonto()	lee el atributo indicado sin modificarlo
setMonto(BigDecimal monto)	cambia el atributo indicado
getCambio()	lee el atributo indicado sin modificarlo
setCambio(BigDecimal cambio)	cambia el atributo indicado
getFechaHora()	lee el atributo indicado sin modificarlo
setFechaHora(LocalDateTime fechaHora)	cambia el atributo indicado

Pedido

src/main/java/com/garagecoffee/model/Pedido.java

Función	Explicación sencilla
Pedido(int id, String observaciones)	constructor: crea y prepara un objeto de esta clase
calcularTotal()	suma los subtotales del pedido
agregarDetalle(DetallePedido detalle)	añade una línea y actualiza el total

Función	Explicación sencilla
cancelar()	cancela según el estado permitido
getId()	lee el atributo indicado sin modificarlo
setId(int id)	cambia el atributo indicado
getFechaCreacion()	lee el atributo indicado sin modificarlo
setFechaCreacion(LocalDateTime fechaCreacion)	cambia el atributo indicado
getEstado()	lee el atributo indicado sin modificarlo
setEstado(EstadoPedido estado)	cambia el atributo indicado
getTotal()	lee el atributo indicado sin modificarlo
setTotal(BigDecimal total)	cambia el atributo indicado
getObservaciones()	lee el atributo indicado sin modificarlo
setObservaciones(String observaciones)	cambia el atributo indicado
getDetalles()	lee el atributo indicado sin modificarlo

PedidoLocal

src/main/java/com/garagecoffee/model/PedidoLocal.java

Función	Explicación sencilla
PedidoLocal(int id, String observaciones, Mesa mesa)	constructor: crea y prepara un objeto de esta clase
getMesa()	lee el atributo indicado sin modificarlo
setMesa(Mesa mesa)	cambia el atributo indicado

Producto

src/main/java/com/garagecoffee/model/Producto.java

Función	Explicación sencilla
Producto()	constructor: crea y prepara un objeto de esta clase
Producto(int id, String nombre, String descripcion, String imagenUrl, BigDecimal precio, boolean disponible, Categoria categoria)	constructor: crea y prepara un objeto de esta clase
getId()	lee el atributo indicado sin modificarlo
getNombre()	lee el atributo indicado sin modificarlo
getDescripcion()	lee el atributo indicado sin modificarlo
getImagenUrl()	lee el atributo indicado sin modificarlo
getPrecio()	lee el atributo indicado sin modificarlo
isDisponible()	responde verdadero o falso sobre ese estado
getcategoria()	lee el atributo indicado sin modificarlo
setId(int id)	cambia el atributo indicado
setNombre(String nombre)	cambia el atributo indicado
setDescripcion(String descripcion)	cambia el atributo indicado
setImagenUrl(String imagenUrl)	cambia el atributo indicado
setPrecio(BigDecimal precio)	cambia el atributo indicado

Función	Explicación sencilla
setDisponible(boolean disponible)	cambia el atributo indicado
ActualizarPrecio(BigDecimal nuevoPrecio)	cambia el precio guardado en el producto
cambiarDisponibilidad()	invierte entre disponible y no disponible
setCategoria(Categoria categoria)	cambia el atributo indicado

AtencionService

src/main/java/com/garagecoffee/service/AtencionService.java

Función	Explicación sencilla
AtencionService(DatosRestaurante datos, SesionService sesion)	constructor: crea y prepara un objeto de esta clase
mesas()	obtiene o devuelve la lista de mesas permitida
guardarMesa(int id, int numero, int capacidad, boolean disponible)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
eliminarMesa(Mesa mesa)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

CatalogoService

src/main/java/com/garagecoffee/service/CatalogoService.java

Función	Explicación sencilla
CatalogoService(DatosRestaurante datos, SesionService sesion)	constructor: crea y prepara un objeto de esta clase
categorias()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
productos()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardarCategoria(int id, String nombre, TipoCategoria tipo, boolean activo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
eliminarCategoria(Categoria categoria)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardarProducto(int id, String nombre, String descripcion, String imagen, BigDecimal precio, Categoria categoria, boolean disponible)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
eliminarProducto(Producto producto)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
precioVigente(Producto producto)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

Claves

src/main/java/com/garagecoffee/service/Claves.java

Función	Explicación sencilla
Claves()	constructor: crea y prepara un objeto de esta clase
protegida(String clave)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
proteger(char[] clave)	crea una huella segura de contraseña
verificar(char[] clave, String guardada)	compara una clave escrita con la almacenada
calcular(char[] clave, byte[] sal, int vueltas)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

DatosMySQL

src/main/java/com/garagecoffee/service/DatosMySQL.java

Función	Explicación sencilla
DatosMySQL()	constructor: crea y prepara un objeto de esta clase
siguienteId(Class<?> tipo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
recargar()	vuelve a consultar datos y refresca la vista
transaccion(Supplier<T> accion)	ejecuta varios cambios como una sola unidad
recuperar()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardar(Object registro)	crea o conserva cambios después de validar
eliminar(Object registro)	borra un registro si sus relaciones lo permiten
llenar(List<T> destino, List<T> origen)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
buscar(List<T> lista, ToIntFunction<T> id, int buscado)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
cargar()	carga datos necesarios para continuar

FinanzasService

src/main/java/com/garagecoffee/service/FinanzasService.java

Función	Explicación sencilla
FinanzasService(DatosRestaurante datos, SesionService sesion, PedidoService pedidos)	constructor: crea y prepara un objeto de esta clase
permiso()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
cajas()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
pagos()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
abierta()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
saldo(Caja caja)	calcula el efectivo esperado
abrir(BigDecimal inicial)	abre el recurso o la caja correspondiente
cerrar(BigDecimal contado)	cierra el recurso o la caja correspondiente
pendiente(Pedido pedido)	calcula cuánto falta pagar
cobrar(Pedido pedido, BigDecimal recibido)	registra efectivo, pago y cambio
exigirCaja()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
ticket(Pedido pedido)	construye el comprobante de texto

PedidoService

src/main/java/com/garagecoffee/service/PedidoService.java

Función	Explicación sencilla
PedidoService(DatosRestaurante datos, SesionService sesion, CatalogoService catalogo)	constructor: crea y prepara un objeto de esta clase
pedidos()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardar(Mesa mesa, String observaciones, List<DetallePedido> detalles)	crea o conserva cambios después de validar
preparar(Pedido pedido)	pasa un pedido pendiente a preparación
listo(Pedido pedido)	marca un pedido preparado como listo
entregar(Pedido pedido)	marca un pedido listo como entregado

Función	Explicación sencilla
cancelar(Pedido pedido)	cancela según el estado permitido
cambiarEstado(Pedido pedido, EstadoPedido esperado, EstadoPedido nuevo, RolEmpleado rol)	aplica el cambio solicitado después de validarlo
exigirEstado(Pedido pedido, EstadoPedido esperado)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
liberarSiPagado(Pedido pedido)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
liberarMesa(Pedido pedido)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

SesionService

src/main/java/com/garagecoffee/service/SesionService.java

Función	Explicación sencilla
SesionService(DatosRestaurante datos)	constructor: crea y prepara un objeto de esta clase
iniciarSesion(String usuario, char[] clave)	verifica usuario, clave, estado y rol
cerrarSesion()	borra el empleado de la sesión actual
actual()	devuelve el empleado autenticado o detiene la operación
permite(RolEmpleado... roles)	responde si el rol puede realizar una acción
exigir(RolEmpleado... roles)	lanza un error cuando una regla no se cumple
listarEmpleados()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardarEmpleado(int id, String nombre, String apellido, String usuario, String clave, String telefono, String email, RolEmpleado rol, boolean activo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardarEmpleadoInterno(int id, String nombre, String apellido, String usuario, String clave, String telefono, String email, RolEmpleado rol, boolean activo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

Validacion

src/main/java/com/garagecoffee/service/Validacion.java

Función	Explicación sencilla
Validacion()	constructor: crea y prepara un objeto de esta clase
exigir(boolean condicion, String mensaje)	lanza un error cuando una regla no se cumple
texto(String valor, String campo, int maximo)	limpia y valida un texto obligatorio
opcional(String valor, String campo, int maximo)	limpia y limita un texto opcional
numero(BigDecimal valor, String campo, int escala, boolean positivo)	valida rango y decimales de un número
dinero(BigDecimal valor, String campo, boolean positivo)	valida una cantidad monetaria con dos decimales

AperturaCajaForm

src/main/java/com/garagecoffee/view/AperturaCajaForm.java

Función	Explicación sencilla
AperturaCajaForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

AtencionPanel

src/main/java/com/garagecoffee/view/AtencionPanel.java

Función	Explicación sencilla
AtencionPanel()	constructor: crea y prepara un objeto de esta clase
AtencionPanel(RestauranteController control)	constructor: crea y prepara un objeto de esta clase
editarMesa(Mesa mesa)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
initComponents()	código visual generado por NetBeans que crea y acomoda controles

CajaPanel

src/main/java/com/garagecoffee/view/CajaPanel.java

Función	Explicación sencilla
CajaPanel()	constructor: crea y prepara un objeto de esta clase
CajaPanel(RestauranteController control)	constructor: crea y prepara un objeto de esta clase
cobrar(Pedido pedido)	registra efectivo, pago y cambio
insertUpdate(javax.swing.event.DocumentEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
removeUpdate(javax.swing.event.DocumentEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
changedUpdate(javax.swing.event.DocumentEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
recargar()	vuelve a consultar datos y refresca la vista
initComponents()	código visual generado por NetBeans que crea y acomoda controles

CantidadForm

src/main/java/com/garagecoffee/view/CantidadForm.java

Función	Explicación sencilla
CantidadForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

CatalogoPanel

src/main/java/com/garagecoffee/view/CatalogoPanel.java

Función	Explicación sencilla
CatalogoPanel()	constructor: crea y prepara un objeto de esta clase
CatalogoPanel(RestauranteController control)	constructor: crea y prepara un objeto de esta clase
producto(Producto p)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
categoria(Categoria c)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
initComponents()	código visual generado por NetBeans que crea y acomoda controles

CategoriaForm

src/main/java/com/garagecoffee/view/CategoriaForm.java

Función	Explicación sencilla
CategoriaForm()	constructor: crea y prepara un objeto de esta clase

Función	Explicación sencilla
initComponents()	código visual generado por NetBeans que crea y acomoda controles

CierreCajaForm

src/main/java/com/garagecoffee/view/CierreCajaForm.java

Función	Explicación sencilla
CierreCajaForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

CobroForm

src/main/java/com/garagecoffee/view/CobroForm.java

Función	Explicación sencilla
CobroForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

CocinaPanel

src/main/java/com/garagecoffee/view/CocinaPanel.java

Función	Explicación sencilla
CocinaPanel()	constructor: crea y prepara un objeto de esta clase
CocinaPanel(RestauranteController control)	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

DashboardFrame

src/main/java/com/garagecoffee/view/DashboardFrame.java

Función	Explicación sencilla
DashboardFrame()	constructor: crea y prepara un objeto de esta clase
DashboardFrame(com.garagecoffee.controller.RestauranteController control)	constructor: crea y prepara un objeto de esta clase
organizarMenu()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
enlace(javax.swing.JButton boton, String nombre, java.util.function.Supplier<PanelModulo> crear, boolean permitido)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
mostrar(String nombre, java.util.function.Supplier<PanelModulo> crear)	abre o actualiza la sección solicitada
initComponents()	código visual generado por NetBeans que crea y acomoda controles

EmpleadoForm

src/main/java/com/garagecoffee/view/EmpleadoForm.java

Función	Explicación sencilla
EmpleadoForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

EmpleadosPanel

src/main/java/com/garagecoffee/view/EmpleadosPanel.java

Función	Explicación sencilla
EmpleadosPanel()	constructor: crea y prepara un objeto de esta clase
EmpleadosPanel(RestauranteController control)	constructor: crea y prepara un objeto de esta clase
editar(Empleado e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
initComponents()	código visual generado por NetBeans que crea y acomoda controles

Formulario

src/main/java/com/garagecoffee/view/Formulario.java

Función	Explicación sencilla
registrar(String etiqueta, JComponent componente)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
campoRegistrado(String etiqueta)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
campo(String etiqueta, T valor)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
etiqueta(String etiqueta, String texto)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
clave(String etiqueta)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
listado(String etiqueta)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
texto(String etiqueta, Object valor)	limpia y valida un texto obligatorio
check(String etiqueta, boolean marcado)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
combo(String etiqueta, List<T> opciones, T actual)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
getListCellRendererComponent(JList<?> lista, Object valor, int indice, boolean elegido, boolean foco)	lee el atributo indicado sin modificarlo
elegido(JComboBox<T> combo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
valor(JTextField campo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
entero(JTextField campo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
decimal(JTextField campo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
fecha(JTextField campo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
fechaHora(JTextField campo)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardar(Component padre, String titulo, Runnable guardar)	crea o conserva cambios después de validar
getPreferredScrollableViewportSize()	lee el atributo indicado sin modificarlo
getScrollableUnitIncrement(Rectangle visible, int orientacion, int direccion)	lee el atributo indicado sin modificarlo
getScrollableBlockIncrement(Rectangle visible, int orientacion, int direccion)	lee el atributo indicado sin modificarlo
getScrollableTracksViewportWidth()	lee el atributo indicado sin modificarlo
getScrollableTracksViewportHeight()	lee el atributo indicado sin modificarlo

InicioPanel

src/main/java/com/garagecoffee/view/InicioPanel.java

Función	Explicación sencilla
InicioPanel()	constructor: crea y prepara un objeto de esta clase
InicioPanel(com.garagecoffee.controller.RestauranteController control)	constructor: crea y prepara un objeto de esta clase
recargar()	vuelve a consultar datos y refresca la vista
initComponents()	código visual generado por NetBeans que crea y acomoda controles

LineaPedidoForm

src/main/java/com/garagecoffee/view/LineaPedidoForm.java

Función	Explicación sencilla
LineaPedidoForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

LoginFrame

src/main/java/com/garagecoffee/view/LoginFrame.java

Función	Explicación sencilla
LoginFrame()	constructor: crea y prepara un objeto de esta clase
LoginFrame(com.garagecoffee.controller.RestauranteController control)	constructor: crea y prepara un objeto de esta clase
ingresar()	intenta iniciar sesión y comunica el resultado
initComponents()	código visual generado por NetBeans que crea y acomoda controles

MesaForm

src/main/java/com/garagecoffee/view/MesaForm.java

Función	Explicación sencilla
MesaForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

PanelListado

src/main/java/com/garagecoffee/view/PanelListado.java

Función	Explicación sencilla
PanelListado()	constructor: crea y prepara un objeto de esta clase
PanelListado(String[] encabezados, java.util.function.Supplier<java.util.List<T>> consulta, java.util.function.Function<T, Object[]> columnas)	constructor: crea y prepara un objeto de esta clase
componentResized(java.awt.event.ComponentEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
getTableCellRendererComponent(javax.swing.JTable t, Object valor, boolean seleccionado, boolean foco, int fila, int columna)	lee el atributo indicado sin modificarlo
filtrar()	aplica el texto de búsqueda al listado
insertUpdate(javax.swing.event.DocumentEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
removeUpdate(javax.swing.event.DocumentEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
changedUpdate(javax.swing.event.DocumentEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código

Función	Explicación sencilla
mouseMoved(java.awt.event.MouseEvent e)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
configurar(String[] encabezados, java.util.function.Supplier<java.util.List<T>> consulta, java.util.function.Function<T, Object[]> columnas)	prepara columnas, consulta y conversión de una tabla Swing
isCellEditable(int fila, int columna)	responde verdadero o falso sobre ese estado
getColumnClass(int columna)	lee el atributo indicado sin modificarlo
boton(String titulo, Runnable accion)	crea un botón y conecta la acción que ejecutará
alternar(String mostrar, String ocultar, java.util.function.Consumer<Boolean> cambio)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
accion(String titulo, java.util.function.Consumer<T> accion)	conecta un botón con el objeto seleccionado
accionesSecundarias(java.util.List<javax.swing.JButton> secundarios)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
ajustarAcciones()	acomoda los botones según el espacio disponible
seleccionado()	devuelve la fila elegida o avisa que falta selección
mensajeVacio(String mensaje)	define el texto que aparece cuando no hay registros
actualizarContador()	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
recargar()	vuelve a consultar datos y refresca la vista
initComponents()	código visual generado por NetBeans que crea y acomoda controles

PanelModulo

src/main/java/com/garagecoffee/view/PanelModulo.java

Función	Explicación sencilla
registrar(PanelListado<?> listado)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
recargar()	vuelve a consultar datos y refresca la vista

PedidoDatosForm

src/main/java/com/garagecoffee/view/PedidoDatosForm.java

Función	Explicación sencilla
PedidoDatosForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

PedidoDialog

src/main/java/com/garagecoffee/view/PedidoDialog.java

Función	Explicación sencilla
PedidoDialog()	constructor: crea y prepara un objeto de esta clase
PedidoDialog(Window padre, RestauranteController control, Mesa elegida)	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

PedidosPanel

src/main/java/com/garagecoffee/view/PedidosPanel.java

Función	Explicación sencilla
PedidosPanel()	constructor: crea y prepara un objeto de esta clase
PedidosPanel(RestauranteController control)	constructor: crea y prepara un objeto de esta clase
detalle(Pedido pedido)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
initComponents()	código visual generado por NetBeans que crea y acomoda controles

ProductoForm

src/main/java/com/garagecoffee/view/ProductoForm.java

Función	Explicación sencilla
ProductoForm()	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

TextoDialog

src/main/java/com/garagecoffee/view/TextoDialog.java

Función	Explicación sencilla
TextoDialog()	constructor: crea y prepara un objeto de esta clase
TextoDialog(java.awt.Window padre, String titulo, String texto)	constructor: crea y prepara un objeto de esta clase
initComponents()	código visual generado por NetBeans que crea y acomoda controles

Ui

src/main/java/com/garagecoffee/view/Ui.java

Función	Explicación sencilla
Ui()	constructor: crea y prepara un objeto de esta clase
prepararApariencia()	configura colores y estilo común de Swing
estiloBoton(JButton boton, Color fondo, Color texto)	responde verdadero o falso sobre ese estado
primario(JButton boton)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
secundario(JButton boton)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
navegacion(JButton boton, boolean seleccionado)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
nombre(Object valor)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
fecha(LocalDateTime fecha)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
dinero(BigDecimal monto)	valida una cantidad monetaria con dos decimales
ejecutar(Component padre, Runnable accion)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
aviso(String texto)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
boton(String texto, Runnable accion)	crea un botón y conecta la acción que ejecutará
mostrarTexto(Component padre, String titulo, String texto)	abre un diálogo para mostrar un documento de texto
areaTexto(String texto)	función auxiliar de esta clase; encapsula una tarea concreta para no repetir código
guardarTexto(Component padre, String texto)	permite exportar el texto a un archivo
imprimir(Component padre, JTextArea area, String titulo)	manda el texto a la impresora

Apéndice B · Glosario sencillo

Palabra	Significado
API	Puerta con rutas para que otro programa pida trabajo.
Bean	Objeto creado y administrado por Spring.
Callback	Función que se ejecuta cuando termina otra tarea.
CRUD	Crear, leer, actualizar y eliminar.
DAO	Clase que conoce el SQL de una tabla.
DTO/record	Ficha pequeña para recibir o devolver JSON.
JDBC	Puente de Java hacia una base SQL.
JSON	Texto estructurado que intercambian Android y Spring.
Hash/huella	Resultado irreversible usado para verificar contraseñas.
Inyección	Spring entrega a una clase los objetos que necesita.
Recomposición	Compose redibuja lo necesario cuando cambia el estado.
Transacción	Grupo de cambios que se guarda completo o se deshace completo.

Idea final: cada capa tiene un trabajo pequeño y claro. La pantalla conversa; el servicio decide; el DAO traduce; MySQL recuerda; Spring conecta; Android acompaña al personal.